

Assignment 01

TensorFlow/Keras Setup, Preprocessing and Fashion-MNIST Classification

Student: Samyak Shende **Roll No.:** 64 **Subject:** Deep Learning

Practical No.: 1 **Academic Year:** 2026–27

1. Problem Statement

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

2. Dataset Used

Fashion-MNIST is loaded using the Keras dataset API. It provides 60,000 training images and 10,000 test images, with each image being 28×28 grayscale and belonging to one of 10 fashion categories.

Official dataset documentation: https://keras.io/api/datasets/fashion_mnist/

3. Required Environment

Use Google Colab with Python, TensorFlow/Keras and Matplotlib. No separate Fashion-MNIST download is required; the dataset is retrieved by `tf.keras.datasets.fashion_mnist.load_data()`.

4. Step-by-Step Execution

Step 1: Open Google Colab and upload/open Assignment-01.ipynb.

Step 2: Run the import cell to load TensorFlow and Matplotlib.

Step 3: Load Fashion-MNIST using the Keras dataset API. The loader provides predefined training and testing partitions.

Step 4: Normalize both image arrays by dividing pixel values by 255.0, converting the input scale from 0–255 to 0–1.

Step 5: Inspect the shapes and labels. The supplied notebook reports 60,000 training images and 10,000 testing images, each of size 28×28.

Step 6: Visualize a sample image using Matplotlib and map its numerical label to a class name.

Step 7: Build a Sequential model: Flatten → Dense(128, ReLU) → Dense(10, Softmax).

Step 8: Compile using Adam, sparse categorical crossentropy and accuracy.

Step 9: Train for 5 epochs.

Step 10: Evaluate on the test set and record loss and accuracy.

Step 11: Generate predictions, select the highest-probability class, and compare it with the actual label.

Step 12: Save the trained model as `fashion_mnist_model.keras` and display the model summary.

5. Algorithm

1. Start the program.
2. Import TensorFlow/Keras and Matplotlib.
3. Load Fashion-MNIST.
4. Use the dataset's training and testing partitions.
5. Normalize pixels to 0–1.
6. Visualize sample data.
7. Build the Sequential neural network.
8. Compile the model.
9. Train for the specified epochs.
10. Evaluate on test data.

11. Predict labels and compare with actual labels.
12. Save the trained model.
13. Stop.

6. Model Architecture

Layer	Configuration	Purpose
Input	28×28 grayscale image	Input sample
Flatten	784 values	Convert image to 1D vector
Dense	128 neurons, ReLU	Learn non-linear features
Dense	10 neurons, Softmax	10-class probability output

7. Important Code

```
import tensorflow as tf
import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) = (
    tf.keras.datasets.fashion_mnist.load_data()
)

train_images = train_images / 255.0
test_images = test_images / 255.0

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_images, train_labels, epochs=5)

loss, accuracy = model.evaluate(test_images, test_labels)
print("Test Accuracy:", accuracy)

predictions = model.predict(test_images)
predicted_class = predictions[0].argmax()

model.save("fashion_mnist_model.keras")
model.summary()
```

8. Output Recorded in the Supplied Notebook

The notebook output reports the following dataset shapes:

```
Training Images: (60000, 28, 28)
Testing Images: (10000, 28, 28)
```

The supplied notebook records a test accuracy of approximately 87.49% after 5 epochs.

```
Test Accuracy: 0.8748999834060669
```

For the first test image, the saved output shows Predicted: Ankle Boot and Actual: Ankle Boot.

9. Important Clarification

The practical statement refers to train-test split, but the submitted implementation does not call `sklearn.model_selection.train_test_split`. Instead, it uses the predefined training and test partitions returned by the Fashion-MNIST Keras loader. This is the accurate way to describe the submitted implementation.

10. Troubleshooting

If the dataset does not load, rerun the loading cell with an active internet connection in Colab. If TensorFlow is unavailable, restart the runtime and rerun the notebook. If the model-saving cell fails, ensure the notebook has write access to the current Colab working directory.

11. Recommended Repository Structure

```
Assignment-01/  
├─ Assignment-01.ipynb  
├─ README.md  
├─ Assignment_01_Fashion_MNIST_Instructions.pdf  
└─ fashion_mnist_model.keras # optional generated file
```

12. Dataset and Documentation Links

Fashion-MNIST: https://keras.io/api/datasets/fashion_mnist/

Keras Getting Started: https://keras.io/getting_started/

Google Colab: <https://colab.research.google.com/>

13. Conclusion

The practical demonstrates an end-to-end introductory TensorFlow/Keras image-classification workflow: dataset loading, preprocessing, normalization, visualization, model construction, training, evaluation, prediction and model saving.

14. Complete Notebook Code

Code Cell 1

```
# Import TensorFlow for building, training, and evaluating the neural network.
# Matplotlib is used to visualize sample images from the dataset.
import tensorflow as tf
import matplotlib.pyplot as plt
```

Code Cell 2

```
# Load the Fashion-MNIST dataset and divide it into training and testing sets.
# The training set is used to learn the model parameters, while the test set
# is used to evaluate how well the trained model performs on unseen images.
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()
```

Code Cell 3

```
# Normalize pixel values from the range [0, 255] to [0, 1].
# This helps the neural network train more efficiently and improves numerical stability.
train_images = train_images / 255.0
test_images = test_images / 255.0
```

Code Cell 4

```
# Inspect the dimensions of the training and testing datasets
# and display the corresponding training labels.
print("Training Images:", train_images.shape)
print("Testing Images:", test_images.shape)
print("First 10 Training Labels:", train_labels[:10])
```

Code Cell 5

```
# Visualize the first image in the training dataset.
# The color bar represents the normalized pixel intensity values.
plt.imshow(train_images[0], cmap='gray')
plt.colorbar()
plt.show()
```

Code Cell 6

```
# Define the class names corresponding to the numerical labels (0-9)
# in the Fashion-MNIST dataset.
class_names = [
    "T-shirt",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle Boot"
]

print(class_names[train_labels[0]])
```

Code Cell 7

```
# Build a Sequential neural network for Fashion-MNIST classification.
# The model processes each 28x28 grayscale image and predicts one of 10 classes.
model = tf.keras.models.Sequential()

# Flatten the 28x28 image into a one-dimensional vector of 784 values.
model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

# Fully connected hidden layer with 128 neurons.
# ReLU activation introduces non-linearity and helps the network learn complex patterns.
model.add(tf.keras.layers.Dense(128, activation='relu'))

# Output layer with 10 neurons, one for each Fashion-MNIST class.
# Softmax converts the outputs into class probabilities.
model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

Code Cell 8

```
# Configure the model for training.
# Adam is used as the optimizer, while sparse categorical crossentropy
# is suitable because the class labels are represented as integers.
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Code Cell 9

```
# Train the neural network using the training images and their corresponding labels.
# The model will pass through the complete training dataset 5 times (epochs).
model.fit(
    train_images,
    train_labels,
```

```
    epochs=5  
)
```

Code Cell 10

```
# Evaluate the trained model on the unseen test dataset.  
# This measures how well the model generalizes to images it did not see during training.  
loss, accuracy = model.evaluate(test_images, test_labels)  
  
print("Test Accuracy:", accuracy)
```

Code Cell 11

```
# Generate class-probability predictions for all test images.  
predictions = model.predict(test_images)  
  
# Display the predicted probabilities for the first test image.  
print(predictions[0])  
  
# Select the class with the highest predicted probability.  
predicted_class = predictions[0].argmax()  
  
print("Predicted:", class_names[predicted_class])  
print("Actual:", class_names[test_labels[0]])
```

Code Cell 12

```
# Save the trained model so that it can be reused later  
# without having to train it again from scratch.  
model.save("fashion_mnist_model.keras")
```

Code Cell 13

```
# Display the architecture of the neural network,  
# including the layers, output shapes, and number of parameters.  
model.summary()
```